SDLC Project Plan: Office Desk & Room Booking Application

A software framework aimed at solving the hybrid office dilemma (space constraints, ghost meetings, and team fragmentation) using the traditional Software Development Life Cycle (SDLC). New additions are marked in green.

Phase 1: Planning & Feasibility

- **Objective:** Establish a centralized reservation system to optimize hybrid workspace usage.
- **Scope Management:** Focus strictly on building a Minimum Viable Product (MVP). High-complexity features like interactive spatial multi-floor maps are deferred to later iterations.

Phase 2: Analysis & Requirements Definition

- **Functional Requirements (User Stories):**
  - As an employee, I want to view available desks for tomorrow so I can secure a seat before commuting.
  - As an employee, I want to reserve a desk so that double-bookings are systematically blocked.

– As an employee, I want my booking to auto-release if I don't check in within 15 minutes, so ghost bookings don't block desks for others.

- **Core Data Requirements:** System must capture data entities for Users (identities), Desks (assets), and Bookings (transactions).
- **Authentication (new):** Login via email/password for MVP; SSO (Google Workspace / Microsoft 365) explicitly out of scope for v1 but noted as a fast-follow.

Phase 3: Product Design

- **System Architecture:** Delivery channel selected — cross-platform responsive web application (chatbot interface deferred).
- **Database Schema:** Define relations and constraints mapping Users to Desks through unique relational Booking timelines.
- **Concurrency Strategy (new):** Enforce a database-level unique constraint on (desk_id, date, timeslot), rather than application-level locking, so simultaneous booking requests fail safely at the DB layer instead of racing in app code.
- **User Interface (UI):** Wireframe foundational screens including Login, Availability Dashboard, and Booking Confirmation states.

Phase 4: Coding & Implementation

- **Tech Stack (finalized):** Node.js + Express backend, PostgreSQL database, React frontend.
- **Development:** Establish database models, implement the DB-level concurrency constraint from Phase 3, build the auto-release job for no-shows, and deploy front-facing components.

Phase 5: Testing & Integration

- **Scenario Testing:** Stress-test concurrency constraints (e.g., handling identical timestamp booking attempts for the same asset).
- **Bug Resolution:** Validate form field states, coordinate timezone alignments for reservation dates, and fix data regressions.

Phase 6: Deployment & Maintenance

- **Deployment Pipeline:** Set up secure cloud infrastructure (such as Vercel, AWS, or Heroku) to make the web endpoint globally accessible.
- **Maintenance Cycles:** Monitor production reliability and plan the next sprint backlog based on real user feature requests.